

TECHNICAL BRIEF · CONFIDENTIAL

Reacti

Application Architecture & Current Hosting Setup

A hand-off document describing how the Reacti messaging app is built and how it is hosted today on a Hostinger VPS.

Product: Reacti - iOS messaging app (live on the App Store), Android-capable

Audience: Cloud / infrastructure architects, and the Reacti technical team

Prepared: June 2026 · **App version:** 1.3.0 · **Stage:** Production, early traction

About this document

Reacti is a live mobile messaging product. Its signature, patent-protected feature is **automatic, silent front-camera recording the moment a recipient opens a media message** - the captured "reaction" is sent straight back to the sender, closing a loop of media → reaction → media.

We are evaluating whether to move the backend off our current host (a Hostinger VPS) to a larger cloud provider such as AWS or Google Cloud, or another managed host. This document is written to be handed to an external architect *and* read by our own team. It has three parts:

Section	What it covers
1 • Application architecture	How the app is built - the mobile client, the API, the data stores, the realtime/push services, and the signature reaction flow. Ends with the platform-level capabilities any new host must provide.
2 • Current hosting	Exactly what runs on the Hostinger VPS today, its size and utilisation, how releases deploy, how data is backed up, and the constraints that matter for a migration.
3 • Open items to confirm	The few remaining figures and a configuration question to resolve before a migration is planned.

Per the brief, this document describes the system and its **requirements** rather than recommending a specific provider - the intent is to give an architect enough to propose a target design. All figures here were read from the live production server in June 2026.

1 · Application architecture

1.1 Stack at a glance

Layer	Technology
Mobile client	Flutter 3.7+ (Dart). State via GetX; HTTP via Dio; realtime over the Pusher protocol via dart_pusher_channels ; push via Firebase Messaging; camera / image_picker / video_player for media & reaction capture. Ships on iOS (com.reacti.app); Android target supported.
Backend API	Laravel 11 (PHP 8.2+, runs on 8.3). REST over HTTPS. Auth via JWT (tymon/jwt-auth) plus an App-Key header gate and social login (Google / Apple / Facebook).
Database	MySQL (one schema, ~25 migrations). Currently very small (~1.2 MB).
Realtime	Self-hosted Laravel Reverb - a WebSocket server running on the VPS under Supervisor (not the hosted Pusher service). Pusher-protocol compatible, so the same client library works.
Cache / queue	Redis (the queue connection is Redis; Redis also available for cache).
Push notifications	Firebase Cloud Messaging (server-side via kreat/laravel-firebase).
Media storage	Uploaded photos, videos and avatars are written to the server's local disk under public/uploads/... (~1.6 GB today). An S3 disk is configured in code but is not the active path.
Email	SMTP via Amazon WorkMail (us-east-1) for OTP / registration / password-reset mail.
Analytics & errors	PostHog (EU region) for product analytics and Sentry for error/performance monitoring, on both client and server. Privacy-first: pseudonymous hashed IDs, opt-out honoured.

1.2 System overview

The mobile app talks to a single Laravel API over HTTPS for all create/read/update operations, and holds an open WebSocket subscription to the **self-hosted Reverb server (on the same VPS)** to receive messages, reactions and presence in real time. Push notifications arrive out-of-band through Firebase even when the app is closed.

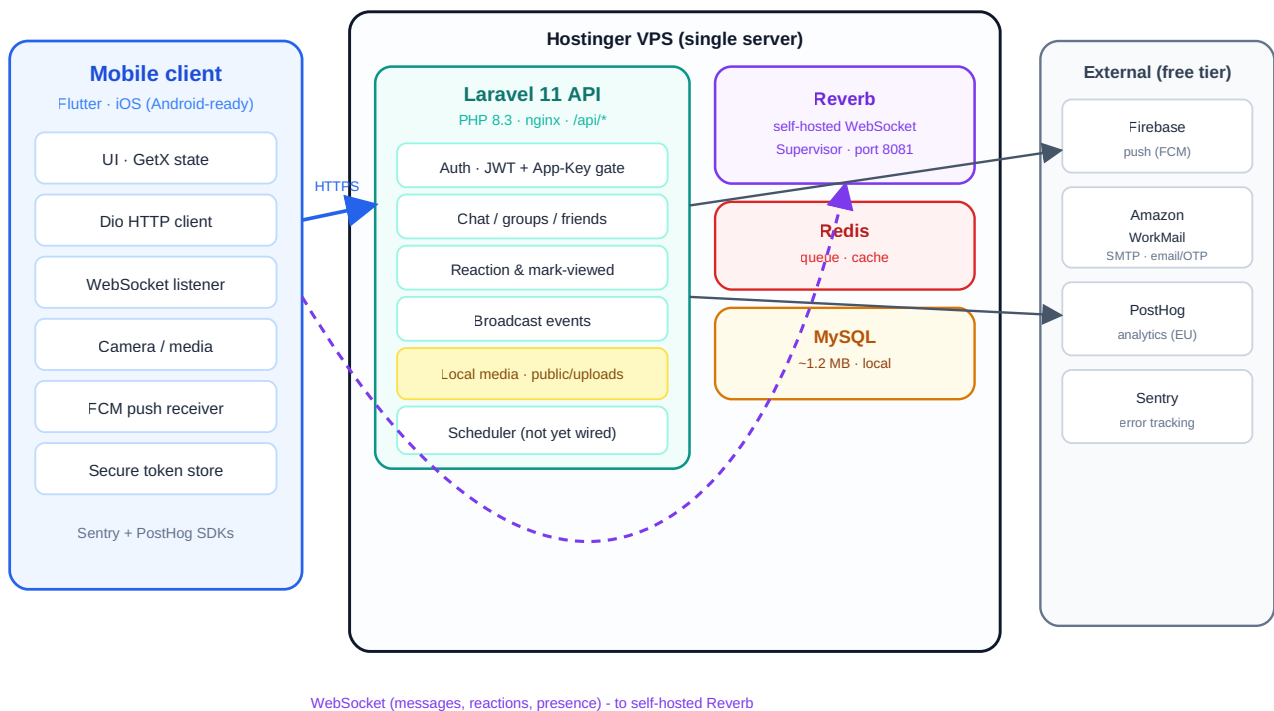


Figure 1 - System overview. Note that realtime (Reverb), Redis and MySQL all run on the VPS itself; only push, email and analytics are external.

1.3 Components in brief

Mobile client (Flutter). One codebase shipping on iOS today and ready for Android. It renders chat, captures photos/videos, performs the silent reaction recording, holds an authenticated WebSocket subscription for live updates, and receives Firebase push when backgrounded. Tokens are kept in secure storage.

API (Laravel 11). A stateless REST API under `/api/*` handling authentication, chat and group messaging, friends/contacts, the reaction and `mark-viewed` endpoints, and broadcasting. Sensitive routes are rate-limited; every request also carries an `App-Key` header gate in addition to the JWT.

Realtime - self-hosted Reverb (important for a migration). Live delivery of messages, reactions and presence runs over **Laravel Reverb**, an open-source WebSocket server, kept alive by **Supervisor** on the VPS (internal port 8083, exposed publicly on 8081). This is a deliberate cost choice - it avoids paying a hosted service like Pusher - but it means the platform must run a **long-running process and expose a dedicated WebSocket port**, which an ordinary PHP web host does not.

Data & cache. A single MySQL schema holds all application data - currently only ~1.2 MB, reflecting early-stage usage. **Redis** backs the queue connection (and is available for cache). The relational data is so small that it migrates in seconds; the real data weight is the media (below).

Media storage - migration-critical. User media and avatars are saved to the application server's **local filesystem** (`public/uploads/...`, ~1.6 GB) and served as static files. The code has an S3 disk configured, but the active upload path writes to local disk. This is the single most important detail for a move: unlike the tiny database, this data lives only on the server.

Push, email & analytics. Firebase Cloud Messaging delivers push. OTP and account email go out over SMTP through **Amazon WorkMail** (us-east-1). PostHog (EU) and Sentry collect product analytics and

errors with privacy controls (hashed IDs, allow-listed events, user opt-out). All of these run on free tiers today.

1.4 The signature flow - silent reaction capture

This is the product's defining, patent-protected loop and the part most sensitive to latency. When a recipient opens a media message, the client calls `mark-viewed`; on success it silently records a short front-camera clip and uploads it back as a `reaction` message, which is broadcast to the original sender in real time through Reverb.

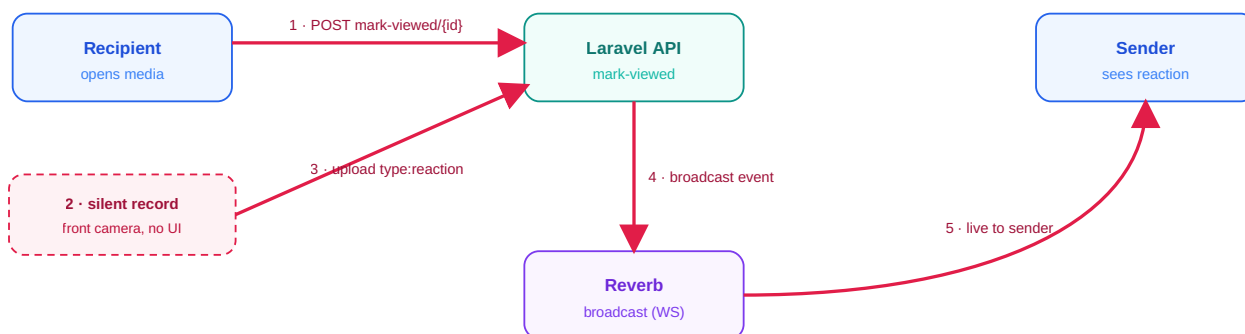


Figure 2 - The reaction capture loop: view → silent record → upload → broadcast → sender.

Latency note for architects: the perceived quality of this feature depends on the round-trip `mark-viewed` → `reaction upload` → `broadcast` being fast. Because realtime (Reverb), the database and the media store currently all sit on the same box, they are co-located today; any target design should preserve low latency between the API, the WebSocket server and the media store.

1.5 What a hosting platform must provide

Capability	Requirement
Application compute	Run PHP 8.2/8.3 (FPM) behind a web server (nginx today). Stateless app tier - can scale horizontally <i>once</i> media moves off local disk.
Realtime (WebSocket)	A long-running process and a public WebSocket port. Reverb is self-hosted today. A migration can either keep self-hosting Reverb (needs a persistent service + a load balancer that supports WebSockets) or switch to a managed service (Pusher / Ably / API Gateway WebSockets). This is the biggest single design decision.
Relational database	Managed MySQL 8 (or compatible). Tiny today (~1.2 MB).
Cache / queue	Redis (managed Redis / ElastiCache / Memystore), used for the queue and cache.
Media / object storage	Durable storage for user photos, videos and avatars (~1.6 GB and growing). Today local disk; a move is the natural moment to shift to object storage (S3 / Cloud Storage), which the code already partly supports.
Background jobs & cron	A queue worker process and a periodic scheduler (cron, once-a-minute). Note: neither is actively configured today (see §2.5) - a clean target setup should add them.

Outbound integrations	HTTPS egress to Firebase (FCM), Amazon WorkMail SMTP, PostHog (EU) and Sentry.
TLS & DNS	TLS for reacti.io and staging.reacti.io , plus the realtime hostname (see §3). Managed certificates.
Two environments	Production and staging, ideally isolated, fed by the existing GitHub Actions pipeline.
Backups & recovery	Automated database <i>and</i> media backups, off-box / cross-region, with a tested restore.

2 · Current hosting - Hostinger VPS

The backend runs on a single **Hostinger VPS** (plan **KVM 4**) managed through **CloudPanel** on Ubuntu 24.04. Both the production site (reacti.io) and a staging site (staging.reacti.io) live on this one server as separate CloudPanel sites under separate Linux users. The server is lightly loaded - it has plenty of head-room at current usage.

4 vCPU ~1% USED	16 GB RAM · ~14% USED	200 GB DISK · 12 GB USED	16 TB BANDWIDTH/MO
---------------------------	---------------------------------	------------------------------------	------------------------------

VPS [srv1153282.hstgr.cloud](https://www.hostinger.com/vps/srv1153282) · IP 72.61.202.136 · backups stored in France (server region EU).

2.1 Server topology

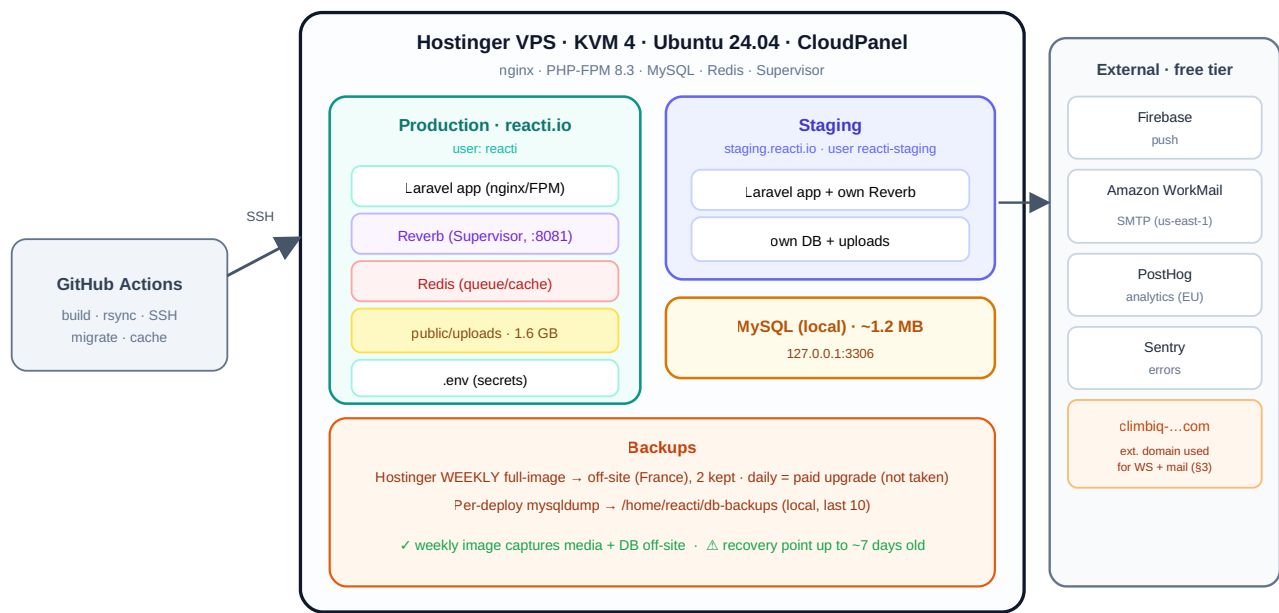


Figure 3 - Current Hostinger VPS topology. One server hosts prod and staging, plus Reverb, Redis, MySQL and media.

2.2 What runs where

Element	Today
Host	Single Hostinger VPS, plan KVM 4 (4 vCPU / 16 GB / 200 GB / 16 TB), Ubuntu 24.04 + CloudPanel.
Web / PHP	nginx + PHP-FPM 8.3 (CloudPanel-managed), running as the site's Linux user.
Realtime	Self-hosted Laravel Reverb under Supervisor (internal :8083, public :8081, https).
Queue / cache	Redis (<code>QUEUE_CONNECTION=redis</code>).
Production	reacti.io at <code>/home/reacti/htdocs/reacti.io/</code> , user <code>reacti</code> .

Staging	<code>staging.reacti.io</code> under <code>reacti-staging</code> ; own DB and uploads. Auto-deploys from <code>develop</code> .
Database	MySQL on the same VPS (<code>127.0.0.1:3306</code>), ~1.2 MB.
Media	Local disk, <code>public/uploads/...</code> (~1.6 GB), served as static files by nginx.
Email	Amazon WorkMail SMTP (<code>smtp.mail.us-east-1.awsapps.com</code>).
External	Firebase (push), PostHog (EU analytics), Sentry (errors) - all free tier, reached over HTTPS.

Cost today: the **only paid infrastructure is the Hostinger VPS itself** - every third-party service (Firebase, PostHog, Sentry, and the self-hosted Reverb instead of paid Pusher) runs on a free tier. The optional Hostinger daily-backup add-on (~\$12/mo) is *not* currently enabled. A migration's cost comparison should therefore weigh a new provider against essentially the VPS line item alone.

2.3 Deployment & release pipeline

- **Production:** a push to `main` triggers a build (Composer + Vite), then an `rsync` over SSH to the VPS, behind a **required-reviewer approval gate**. On the server it runs a best-effort `mysqldump` , then `migrate --force` , rebuilds Laravel caches, restarts the queue, and reloads PHP-FPM.
- **Staging:** a push to `develop` auto-deploys to `staging.reacti.io` as an unprivileged user, with no approval gate.
- After a prod deploy, a smoke-test workflow runs against the live URL; the matching iOS build then ships via TestFlight → App Store.
- Secrets live in GitHub Actions and the server-side `.env` (never in the repo). The `.env` , `storage/` and uploads are excluded from `rsync` so server state is preserved.

Good news for a migration: deploys are already scripted and the app is configured entirely through environment variables, so re-pointing the pipeline at a new host is mostly new SSH targets (or a container build) plus moving the `.env` values - not re-architecting the application.

2.4 Backups & data durability

Two backup mechanisms exist: Hostinger's **weekly full-server image**, stored off-site in France (two retained), and a **per-deploy `mysqldump`** kept locally in `/home/reacti/db-backups` (last ten).

Off-site coverage exists, but coarse. The weekly image does capture both media and database off the server - so this is better than "media is unprotected." The gap is granularity: backups are **weekly** (recovery point up to ~7 days old) unless the paid daily add-on is enabled, and the local DB dumps sit on the same machine they protect.

For the target design: aim for automated *daily* (or continuous) DB backups plus media on durable object storage with versioning, and a documented, tested restore.

2.5 Migration-relevant constraints

- **Self-hosted realtime.** Reverb runs on the VPS - the new platform must support a persistent process and a public WebSocket port (or you switch to a managed WS service). This is the main architectural decision of the move.
- **Single server, two environments.** Prod and staging share one box - convenient but not isolated; a move is a chance to separate them.
- **Stateful app tier.** Local-disk media blocks running more than one app instance; moving to object storage unlocks horizontal scaling.
- **No active queue worker / scheduler.** Redis is the queue backend but no `queue:work` runs under Supervisor and no `schedule:run` cron exists; broadcasting appears to be synchronous today. A clean target should wire both explicitly.
- **External-domain dependency (see §3).** Realtime and email both reference `climbiq-goonclimbers.com`, a non-Reacti domain - to be untangled before/during the move.
- **One pending item:** a deferred Laravel framework security upgrade is tracked separately - noted for completeness, not a migration blocker.

3 · Open items to confirm

Most of the picture is now nailed down from the live server. A short list remains before a migration is planned:

1 · The realtime / email domain - **resolve first**

The Reverb realtime host is set to `REVERB_HOST=climbiq-goonclimbers.com`, while Supervisor advertises `-- public-host=reacti.io`. The same `climbiq-goonclimbers.com` domain is the Amazon WorkMail organisation that sends OTP email. So **both live-chat websockets and OTP email currently depend on a domain that is not Reacti's** (most likely the developer's/agency's).

Act